

```

1  /* =====
2      geo-projektion.js – Georeferenz und Projektion Grad → Bildschirm
3
4      Unterste Schicht: rechnet Länge/Breite in Canvas-Pixel um und bestimmt den
5      Bildausschnitt zu einer Bbox. Alles auf der Karte geht durch
6      lonLatToScreen(). Bewusst lineare Abbildung statt echter Mercator-
7      Projektion – über einen Pariser Ausschnitt ist der Fehler vernachlässigbar.
8  ===== */
9
10 // ACHTUNG die beiden Übersichtskarten zeigen NICHT denselben Ausschnitt:
11 // gleiche Pixelmasse, aber um 568 m verschobene QGIS-Fenster. Je eigene Bbox
12 // nötig – mit gemeinsamen Werten lagen die Routen 46 px neben den Strassen.
13 // Hergang und Gegenprobe: docs/bugfix-log.md, Fix 2.
14
15 // paris-startkarte-web.png (Startseite und Schlusskarte).
16 // QGIS EPSG:3857: X 247340.000 .. 270290.000, Y 6245109.800 .. 6256840.000
17 let startBbox = { west: 2.221893023741224, east: 2.4280563814466545, south: 48.82435089471847,
18   north: 48.89367804058055 };
19
20 // paris-ueberblickkarte-web.png (Übersichtsakt), Basis wie die Kapitelkarten.
21 // QGIS EPSG:3857: X 247907.651 .. 270857.651, Y 6244994.107 .. 6256724.107
22 let uebersichtBbox = { west: 2.2269923194085774, east: 2.4331556771226127, south: 48.82366665448583,
23   north: 48.892993566082404 };
24
25 // Schnittmenge beider Georeferenzen. Der Ausschnitt vor dem Zoom bleibt darin,
26 // damit der Crossfade beide Bilder deckungsgleich zeigt.
27 const UEBERSICHT_SCHNITT_BBOX = {
28   west: Math.max(startBbox.west, uebersichtBbox.west),
29   east: Math.min(startBbox.east, uebersichtBbox.east),
30   south: Math.max(startBbox.south, uebersichtBbox.south),
31   north: Math.min(startBbox.north, uebersichtBbox.north),
32 };
33
34 // kapitel01-qgis-karte-web.png. Wert ist geprüft und korrekt, aber sein
35 // QGIS-Ursprung ist nicht überliefert.
36
37 // ACHTUNG die .pgw-Weltdateien aus data-prep/export gehören NICHT zu diesem
38 // Bild – eingesetzt zögen sie die Route 1.5 km nach Osten. Siehe
39 // docs/cleanup-log.md, Schritt 10.
40 let ch1ImgBbox = { west: 2.317834413581757, east: 2.352393886019969, south: 48.86683338890839,
41   north: 48.881871498351956 };
42
43 let mapOffsetX = -250;
44 let mapOffsetY = 0;
45
46 function coverCrop(imgW, imgH, vAnchor = 0.5, hAnchor = 0.5, offsetX = mapOffsetX) {
47   // Effektive Breite (width – offsetX), sonst bleibt rechts ein grauer Rand.
48   // Die zentrierte Übersichtskarte ruft explizit mit offsetX=0 auf.
49   let canvasRatio = (width – offsetX) / height;
50   let imgRatio = imgW / imgH;
51   let coverW, coverH;
52   if (imgRatio > canvasRatio) { coverH = imgH; coverW = imgH * canvasRatio; }
53   else { coverW = imgW; coverH = imgW / canvasRatio; }
54   // vAnchor/hAnchor verschieben den Ausschnitt im Bild: 0 = oben/links,
55   // 1 = unten/rechts, 0.5 = zentriert.
56   return { x: (imgW – coverW) * hAnchor, y: (imgH – coverH) * vAnchor, w: coverW, h: coverH };
57 }
58
59 function lonLatToScreen(lon, lat, bbox, offsetX = mapOffsetX, offsetY = mapOffsetY) {
60   let x = map(lon, bbox.west, bbox.east, 0, width – offsetX) + offsetX;
61   let y = map(lat, bbox.north, bbox.south, 0, height) + offsetY;
62   return { x, y };
63 }
64
65 function bboxToImgCrop(bbox, refBbox, imgW, imgH) {
66   let x0 = map(bbox.west, refBbox.west, refBbox.east, 0, imgW);

```

```
64   let y0 = map(bbox.north, refBbox.north, refBbox.south, 0, imgH);
65   let x1 = map(bbox.east, refBbox.west, refBbox.east, 0, imgW);
66   let y1 = map(bbox.south, refBbox.north, refBbox.south, 0, imgH);
67   x0 = constrain(x0, 0, imgW); y0 = constrain(y0, 0, imgH);
68   x1 = constrain(x1, 0, imgW); y1 = constrain(y1, 0, imgH);
69   return { x: x0, y: y0, w: Math.max(1, x1 - x0), h: Math.max(1, y1 - y0) };
70 }
71
72 function cropToBbox(crop, refBbox, imgW, imgH) {
73   return {
74     west: map(crop.x, 0, imgW, refBbox.west, refBbox.east),
75     east: map(crop.x + crop.w, 0, imgW, refBbox.west, refBbox.east),
76     north: map(crop.y, 0, imgH, refBbox.north, refBbox.south),
77     south: map(crop.y + crop.h, 0, imgH, refBbox.north, refBbox.south),
78   };
79 }
80
81 // Grösster Ausschnitt mit dem Seitenverhältnis von bbox, der ganz in rahmen
82 // liegt: erst gleichmässig verkleinern, dann hineinschieben.
83 function passeBboxInRahmen(bbox, rahmen) {
84   let breite = bbox.east - bbox.west;
85   let hoehe = bbox.north - bbox.south;
86   let f = Math.min(1, (rahmen.east - rahmen.west) / breite, (rahmen.north - rahmen.south) / hoehe);
87   let mitteX = (bbox.west + bbox.east) / 2;
88   let mitteY = (bbox.south + bbox.north) / 2;
89   breite *= f;
90   hoehe *= f;
91   let b = { west: mitteX - breite / 2, east: mitteX + breite / 2, south: mitteY - hoehe / 2, north:
mitteY + hoehe / 2 };
92   // Nur so weit schieben, wie eine Kante übersteht.
93   let dx = Math.max(0, rahmen.west - b.west) - Math.max(0, b.east - rahmen.east);
94   let dy = Math.max(0, rahmen.south - b.south) - Math.max(0, b.north - rahmen.north);
95   return { west: b.west + dx, east: b.east + dx, south: b.south + dy, north: b.north + dy };
96 }
```